

Vector Search at Scale

Why brute force fails, recall@k, IVF clustering, LSH hashing, HNSW graph search, and Product Quantization

Concept Guide 8 of 9 | Read before: [Vector Search at Scale Zero-to-Hero lab](#) | Companion to the Zero-to-Hero series

1. Why Brute Force Doesn't Scale

Brute-force search compares a query against EVERY vector in the database — cost grows linearly ($O(n)$) with database size. Fine at 1,000 vectors; too slow at 100 million.

DIAGRAM — Linear cost growth

n=1,000		(fast)
n=1,000,000		(noticeable)
n=100,000,000		(too slow for real-time)

MENTAL MODEL

Approximate Nearest Neighbor (ANN) algorithms are like a library's catalog system: instead of reading every book, you use an index to jump straight to the likely shelf, occasionally missing the single best book in exchange for massive speed.

PRACTICE THIS → [Vector_Search_at_Scale_Zero_to_Hero.ipynb Ch.1-2](#)

2. Recall@k: Measuring the Approximation's Cost

Every ANN method trades a small amount of accuracy for speed. Recall@k measures exactly how much accuracy is lost.

FORMULA — Recall@k

$$\text{recall@k} = \frac{|\text{approximate_top_k} \cap \text{exact_top_k}|}{|\text{exact_top_k}|}$$

1.0 = perfect match with brute-force ground truth; lower = some true neighbors were missed

WORKED EXAMPLE — Worked example

exact top-5 (brute force): {42, 17, 8, 99, 3}

approximate top-5 (ANN): {42, 17, 8, 50, 3} (found 4 of 5, missed 99, found 50 instead)

$\text{recall@5} = |\{42, 17, 8, 3\}| / 5 = 4/5 = 0.8$

PRACTICE THIS → [Vector_Search_at_Scale_Zero_to_Hero.ipynb Ch.2, Ch.8](#)

3. IVF: Clustering-Based Search

IVF (Inverted File Index) pre-clusters all vectors with k-Means, then at search time only searches within the FEW clusters nearest to the query — skipping everything else entirely.

DIAGRAM — IVF search

BUILD: cluster all vectors, remember each cluster's centroid
[cluster A: ***] [cluster B: ***] [cluster C: ***]
SEARCH: find nearest centroid(s) to the query, e.g. cluster B
→ only compare against cluster B's vectors (skip A and C)

MENTAL MODEL

n_{probe} (how many clusters to search) is the main speed/accuracy knob: $n_{\text{probe}}=1$ is fastest but riskiest — a query near a cluster boundary might have its true nearest neighbor in the NEIGHBORING cluster. Higher n_{probe} recovers accuracy.

PRACTICE THIS → [Vector_Search_at_Scale_Zero_to_Hero.ipynb Ch.3](#)

4. LSH: Locality-Sensitive Hashing

LSH uses random hash functions specially designed so that SIMILAR vectors are likely to land in the same bucket — the opposite of normal hashing, which deliberately scatters similar items.

FORMULA — Random hyperplane hashing

$$\text{hash_bit}(v) = \text{sign}(\text{hyperplane} \cdot v)$$

which side of a random hyperplane a vector falls on becomes one bit of its hash

```

hyperplane
|
* A      |      * B      A and C are on the SAME side -> same hash bit
|         |      B is on the OTHER side -> different hash bit
|
* C      |

```

5. HNSW: The Industry-Standard Graph Search

DIAGRAM — Navigating layers like highways then local streets

Layer 2 (sparse, long jumps):	A -----> D	'highways'
Layer 1 (medium):	A -> B -> C -----> D -> E	'main roads'
Layer 0 (dense, all nodes):	A-B-C-F-G-H-I-D-J-K-L-E-M-N	'local streets'

search: start at the top layer, greedily move toward the query,
drop a layer when progress stalls, repeat

HSW's key advantage is roughly LOGARITHMIC search time ($O(\log n)$) instead of linear $O(n)$ — this is exactly why it scales to billions of vectors while brute force cannot.

- A greedy graph walk can get stuck in a LOCAL optimum — a node that looks best among its immediate neighbors but isn't the true global best. Real HNSW mitigates this with multiple layers and, in practice, exploring several candidate paths at once (a 'beam' rather than a single greedy path).

6. Product Quantization: Compressing Vectors

Storing millions of full-precision vectors costs a lot of memory. Product Quantization (PQ) splits each vector into pieces, clusters each piece's space separately, and stores just a small codebook INDEX per piece instead of full floats.

DIAGRAM — Splitting and quantizing

original vector (8 floats): [0.2, -0.5, 0.8, 0.1, -0.3, 0.9, 0.4, -0.1]
 sub1 sub2 sub3 sub4
each sub-vector -> nearest codebook centroid -> store just that ID
compressed: [code_3, code_7, code_2, code_5] <- tiny compared to 8 floats

COMMON MISUNDERSTANDINGS

- PQ is LOSSY compression — the reconstructed vector is only an approximation, a real accuracy cost traded for memory savings. Finer splits (more sub-vectors) generally reduce reconstruction error at the cost of a slightly larger code.

PRACTICE THIS → [Vector_Search_at_Scale_Zero_to_Hero.ipynb Ch.6-7](#)

7. Choosing an Index

A practical decision framework for real projects.

DIAGRAM — Rule of thumb

< 10K vectors, exact needed	-> Brute force
medium scale, general default	-> HNSW
huge scale, RAM-constrained	-> IVF + PQ (FAISS's IndexIVFPQ)
frequently updated/streaming data	-> HNSW (handles incremental inserts well)

PRACTICE THIS → [Vector_Search_at_Scale_Zero_to_Hero.ipynb Ch.9-10](#)